Web Scraping con Python

Beautiful Soup y Selenium

Módulo 2: Extracción de Datos Web

Fundamentos

¿Qué es HTML?

HTML (Hypertext Markup Language) es el lenguaje utilizado para crear y estructurar páginas web.

Elementos básicos:

- <html>: Elemento raíz
- <head>: Información meta
- <body>: Contenido visible
- **<h1>-<h6>: Encabezados**
- <p>: Párrafos
- <a>: Enlaces
- <img>: Imágenes

Elementos de tablas:

- <table>: Crea tabla
- <tr>: Filas
- <td>: Celdas de datos
- <th>: Celdas de encabezado

¿Qué es el Web Scraping?

El **web scraping** es una técnica para extraer información de sitios web de forma automática.

Objetivos principales:

- Analizar el código HTML/XML de una página web y extraer datos relevantes de manera estructurada.

Beautiful Soup

Beautiful Soup es una biblioteca de Python para analizar y extraer datos de archivos HTML y XML.

Instalación:

```
pip install beautifulsoup4
```

Métodos Principales

Constructor:

```
from bs4 import BeautifulSoup
soup = BeautifulSoup(html_doc, 'html.parser')
```

Métodos de búsqueda:

- `find()`
- `find_all()`
- `select()`

Extracción de datos:

- `.text`
- `.get()`
- `.attrs`

Beautiful Soup: Flujo de Trabajo

1. Importar bibliotecas:

```
import requests
from bs4 import BeautifulSoup
```

2. Realizar solicitud HTTP:

```
response = requests.get(url)
```

3. Crear objeto BeautifulSoup:

```
soup =
BeautifulSoup(response.content,
"html.parser")
```

4. Encontrar y extraer elementos:

```
elemento = soup.find("tag", class_="clase")
texto = elemento.text
```

Beautiful Soup: Ejemplo Práctico

Extracción de enlaces de una página:

```
import requests
from bs4 import BeautifulSoup

# Realizar solicitud
url = "https://www.example.com"
response = requests.get(url)

# Crear objeto BeautifulSoup
soup = BeautifulSoup(response.content, "html.parser")

# Encontrar todos los enlaces
enlaces = soup.find_all('a')

# Extraer URLs
for enlace in enlaces:
    print(enlace.get('href'))
```

Selenium: Introducción

Selenium es una herramienta de automatización que permite controlar navegadores web y simular acciones de usuario. Selenium abrirá una ventana de Chrome controlada mediante código.

Instalación:

```
pip install selenium
pip install webdriver-manager
```

Configuración básica:

```
from selenium import webdriver
from webdriver_manager.chrome import
ChromeDriverManager

options = webdriver.ChromeOptions()
driver = webdriver.Chrome(
    ChromeDriverManager().install(),
    options=options
)
```

Selenium: Métodos Principales

Navegación:

- `get(url)`
- `back()`
- `forward()`

Búsqueda de elementos:

- `find_element()`
- `find_elements()`

Interacción:

- `click()`
- `send_keys()`

Control del navegador:

- `close()`
- `quit()`

Selenium: Interacción Básica

01

Abrir una página:

```
driver.get('https://www.example.com')
```

02

Buscar elementos:

```
elemento = driver.find_element('id',
'elemento_id')
```

03

Hacer clic:

```
boton.click()
```

04

Escribir texto:

```
campo_input.send_keys('Texto a ingresar')
```

05

Cerrar navegador:

```
driver.close()
```

Selenium: Ejemplo Práctico

Extracción de datos de una página web:

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from webdriver_manager.chrome import ChromeDriverManager

# Configurar el driver
options = webdriver.ChromeOptions()
driver = webdriver.Chrome(
    ChromeDriverManager().install(),
    options=options
)

# Abrir página web
driver.get('https://www.example.com')

# Encontrar elemento y extraer texto
elemento = driver.find_element(By.TAG_NAME, 'h1')
print('Texto extraído:', elemento.text)

# Cerrar navegador
driver.close()
```

Extracción de Datos con Selenium

Selenium permite extraer datos de sitios web dinámicos que requieren interacción.

Casos de uso:

- **Scroll infinito**
- **Login requerido**
- **Contenido JavaScript**

Ventaja clave:

Puede interactuar con la página como un usuario real antes de extraer los datos.

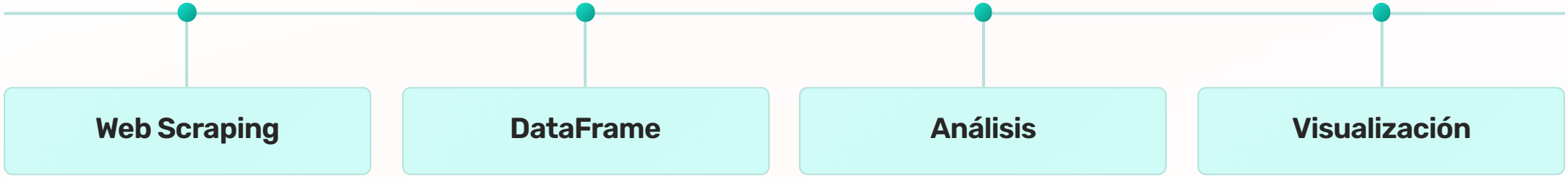
Beautiful Soup vs Selenium

Característica	Beautiful Soup	Selenium
Tipo	Parser HTML/XML	Automatización navegador
Velocidad	Rápido	Más lento
Contenido dinámico	No	Sí
JavaScript	No ejecuta	Ejecuta JavaScript
Interacción	No interactiva	Totalmente interactiva
Uso ideal	Páginas estáticas	Páginas dinámicas

Conclusión: Son herramientas complementarias. Elige según el tipo de página web.

Integración con Pandas

Los datos extraídos pueden analizarse con Pandas:



Flujo completo: Web Scraping → DataFrame → Análisis → Visualización